Topics of interests

Agenda

- What is Prompt Engineering?
- Importance of Prompt Engineering
- Key Strategies for Effective Prompts
- Steps in Prompt Engineering
- Examples of Effective Prompts
- Common Challenges
- Best Practices
- Q&A

Prompt Engineering

What is Prompt Engineering?

Prompt engineering is the process of crafting clear instructions to guide AI systems, like GitHub Copilot, to generate context-appropriate code tailored to your project's specific needs.

- Copilot

Benefits of Well-Engineered Prompts

- Maximises utility of Copilot
- Minimises repeated iterations
- Determines code suggestion quality
- Guides Copilot to understand context & nuances
- Refined interactions reduces misunderstandings

Principles of Prompt Engineering

🎯 SINGLE - Focus on One Task
❌ Bad: "Create a service and tests and documentation"
✅ Good: "Create a LoanService class with async methods"

🔍 SPECIFIC - Explicit Details
❌ Bad: "Use good practices"
✅ Good: "Use async/await, dependency injection, and repository pattern"

Principles of Prompt Engineering

📏 SHORT - Concise but Complete
❌ Bad: 500-word essay
✅ Good: 2-3 sentences with key requirements

🌍 SURROUND - Provide Context
❌ Bad: "Generate a method"
✅ Good: "#file:LoanService.cs Generate ExtendLoan method"

Best Practices for Effective Prompts

- Provide enough clarity: Begin with a broad description and then list specific requirements.
- Give Examples: Use examples to help Copilot understand what you want.
- Break Complex Tasks into Simpler Tasks: Divide large tasks into smaller, manageable tasks.
- Provide enough context with details: The more contextual information provided, the more fitting the generated code suggestions are.
- Assert and iterate: Unlocking GitHub Copilot's full potential with the practice of iteration

Techniques

Zero-Shot Prompting

No Example

One-Shot Prompting

One Example

Few-Shot Prompting

Handful of Examples

Techniques

Zero-Shot Learning with Copilot

abelberhane

Write me unit tests for all of the operations in my calculator.js file.

GitHub Copilot

Sure! Here's an example of a test suite that covers all of the operations in a `calculator.js` file:

```
const { calculate } = require('./calculator');

describe('calculator', () => {
  describe('addition', () => {
    it('adds 1 + 2 to equal 3', () => {
      expect(calculate(1, 2, '+')).toBe(3);
    });

    it('adds -5 + 7 to equal 2', () => {
      expect(calculate(-5, 7, '+')).toBe(2);
    });

    it('adds 0.1 + 0.2 to equal 0.3', () => {
      expect(calculate(0.1, 0.2, '+')).toBeCloseTo(0.3);
    });
  });

  describe('subtraction', () => {
    it('subtracts 5 - 3 to equal 2', () => {
      expect(calculate(5, 3, '-')).toBe(2);
```

Techniques

One-shot Learning with Copilot

abelberhane

Write me unit tests for all of the operations in my calculator.js file.

Use the following example: test('adds 1 + 2 to equal 3', () => { expect(calculate(1, 2, '+')).toBe(3); });

GitHub Copilot

Here are some examples of unit tests for all of the operations in a `calculator.js` file:

```
const { calculate } = require('./calculator');

test('adds 1 + 2 to equal 3', () => {
  const result = calculate(1, 2, '+');
  expect(result).toBe(3);
});

test('subtracts 5 - 3 to equal 2', () => {
  const result = calculate(5, 3, '-');
  expect(result).toBe(2);
});

test('multiplies 2 * 4 to equal 8', () => {
  const result = calculate(2, 4, '*');
  expect(result).toBe(8);
});

test('divides 10 / 5 to equal 2', () => {
  const result = calculate(10, 5, '/');
  expect(result).toBe(2);
});

test('throws an error for an invalid operation', () => {
  expect(() => calculate(1, 2, '%')).toThrow('Invalid operation');
});

test('throws an error for division by zero', () => {
  expect(() => calculate(1, 0, '/')).toThrow('Division by zero');
});
```

Techniques

Few-shot Learning with Copilot

greeting.py

```python
# Example 1: Greeting message for morning
# Input: 9 AM
# Output: "Good morning!"

# Example 2: Greeting message for afternoon
# Input: 2 PM
# Output: "Good afternoon!"

# Example 3: Greeting message for evening
# Input: 7 PM
# Output: "Good evening!"

# Now, generate a python code that takes the current time as input using the datetime module
# and returns the appropriate greeting message

# Solution:
# Import datetime module
import datetime

# Get current time
current_time = datetime.datetime.now()

# Get current hour
current_hour = current_time.hour

# Check if it is morning (before 12 PM)
if current_hour < 12:
    print("Good morning!")

# Check if it is afternoon (between 12 PM and 4 PM)
elif current_hour < 16:
    print("Good afternoon!")

# Check if it is evening (after 4 PM)
elif current_hour < 21:
    print("Good evening!")

# Else it is night time
else:
    print("Good night!")
```

Techniques

Cue
with Copilot

/explain #file:Program.cs The purpose of this file is

Program.cs Program.cs:1-30

GitHub Copilot

Used 1 reference

The purpose of the Program.cs file is to set up and run a .NET application that ingests data from GitHub using HTTP clients. Here's a detailed explanation:

Using Directives

These bring in necessary namespaces for HTTP handling, dependency injection, and hosting:

```
using System.Net.Http.Headers;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.CopilotDashboard.DataIngestion.Services;
```

Host Configuration

A HostBuilder is used to create and configure a host:

```
var host = new HostBuilder()
    .ConfigureFunctionsWebApplication()
    .ConfigureServices((ctx, services) =>
    {
        services.Configure<GithubMetricsApiOptions>(ctx.Configuration.GetSect
        services.AddHttpClient<GitHubCopilotMetricsClient>(ConfigureClient);
        services.AddHttpClient<GitHubCopilotUsageClient>(ConfigureClient);
        services.AddHttpClient<GitHubCopilotApiService>(ConfigureClient);
    })
    .Build();
```

Techniques

Supporting Content with Copilot

shin-akuma

create a powershell method that inserts a json file to cosmosdb collection.

test.ps1:230-258

GitHub Copilot

Used 1 reference

Here is a PowerShell method that inserts a JSON file into a CosmosDB collection:

```
function Insert-JsonToCosmosDb {
    param (
        [string]$cosmosDbEndpoint,
        [string]$cosmosDbKey,
        [string]$databaseName,
        [string]$collectionNam
        [string]$jsonFilePath
    )

    # Install the Az.CosmosDB
    if (-not (Get-Module -List
        Install-Module -Name A
        CurrentUser
    }

    # Import the Az.CosmosDB m
    Import-Module Az.CosmosDB

    # Read the JSON file
    $jsonContent = Get-Content

    # Convert JSON content to a PowerShell object
    $jsonObject = $jsonContent | ConvertFrom-Json
```

shin-akuma

@github create a powershell method that inserts a json file to cosmosdb collection. Use this repo as reference https://github.com/PlagueHO/CosmosDB

get-changed-modules.ps1:24-47

GitHub

Used 7 references

get-changed-modules.ps1 scripts\azure-devops

https://github.com/PlagueHO/CosmosDB/blob/9060ce4dacd08b8e1a119349fb193e05b9acc263/RE...

https://github.com/PlagueHO/CosmosDB/blob/9060ce4dacd08b8e1a119349fb193e05b9acc263/C...

https://github.com/PlagueHO/CosmosDB/blob/9060ce4dacd08b8e1a119349fb193e05b9acc263/so...

https://github.com/PlagueHO/CosmosDB/blob/9060ce4dacd08b8e1a119349fb193e05b9acc263/so...

https://github.com/PlagueHO/CosmosDB/blob/9060ce4dacd08b8e1a119349fb193e05b9acc263/so...

shin-akuma

@github create a powershell method that inserts a json file to cosmosdb collection. Use this repo as reference https://github.com/PlagueHO/CosmosDB

get-changed-modules.ps1:24-47

GitHub

Used 7 references

Repo not indexed for semantic code search

Do you wish to index the repo for semantic code search?

Based on the retrieved information, here's a PowerShell method that inserts a JSON file to a CosmosDB collection using the CosmosDB PowerShell module:

```
$cosmosDbContext = New-CosmosDbContext -Account $AccountName -ResourceGrou

# Read the JSON file
$jsonContent = Get-Content -Path $JsonFilePath -Raw | ConvertFrom-Json
```

Techniques

Provide enough context with details

even_numbers.py

even_numbers.py

```
1  # write a simple flask app that returns a list of even numbers from a list of numbers
2  # Create a function that takes a list of numbers and returns only the even values.
3  # create a sample list of numbers
4  # create a list of even numbers from the sample list
5  # return the list of even numbers
6
7
```

Techniques

Start with clear instructions with Copilot

shin-akuma

@workspace /explain #file:Program.cs line 10

Program.cs Program.cs:10

Workspace

Used 1 reference

The GithubMetricsApiOptions class is used to configure settings for the GitHub Metrics API. It is being configured in the ConfigureServices method using the application's configuration.

Here's an example of what the GithubMetricsApiOptions class might look like:

```
namespace Microsoft.CopilotDashboard.DataIngestion.Services
{
    public class GithubMetricsApiOptions
    {
        public string BaseUrl { get; set; }
        public string ApiVersion { get; set; }
        public string Token { get; set; }
    }
}
```

In the Program.cs file, the GithubMetricsApiOptions class is configured like this:

```
services.Configure<GithubMetricsApiOptions>(ctx.Configuration.GetSection("GITHUB_METRICS"));
```

This line reads the configuration section named "GITHUB_METRICS" from the application's configuration (e.g., `appsettings.json` or environment variables) and binds it to the GithubMetricsApiOptions class. This allows you to inject `IOptions<GithubMetricsApiOptions>` into your services to access these settings.

Example Configuration in `appsettings.json`:

shin-akuma

@github #file:Program.cs #file:Githubl

GithubMetricsApiOptions.cs Program.

GitHub

Used 2 references

The `Program.cs` file sets up dependency
`GithubMetricsApiOptions.cs` file defin
Specifically, in `Program.cs`, the `GithubM`
"GITHUB_METRICS" section of the applic

Techniques

Break the task down with Copilot

Write a function to generate a 10 by 10 grid of letters.

Write a function to find all words in a grid of letters, given a list of valid words.

Write a function that uses the previous functions to generate a 10 by 10 grid of letters that contains at least 10 words.

Update the previous function to print the grid of letters and 10 random words from the grid.

Techniques

Getting Specific with Copilot

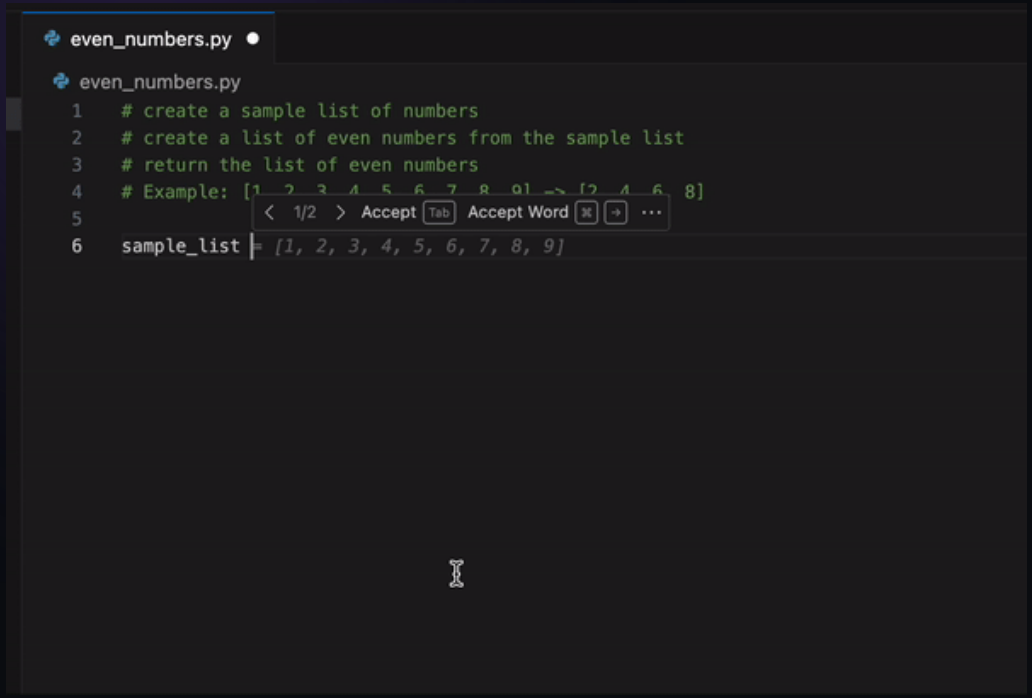
Write a function that tells me if a number is prime

The function should take an integer and return true if the integer is prime

The function should error if the input is not a positive integer

Techniques

Provide examples for learning



Role Prompting

Tell Copilot to assume expertise in a specific domain

🏦 "You are a senior C# developer at Mercury specializing in loan processing systems using Entity Framework Core and async patterns."

Use for: Generating service classes, business logic

🧪 "You are a Mercury QA engineer who writes comprehensive xUnit tests with NSubstitute mocking following the AAA pattern."

Use for: Test generation

🔒 "You are a Mercury security auditor reviewing code for SQL injection, XSS, and OWASP Top 10 vulnerabilities."

Use for: Security reviews

Role Prompting

Why This Works:

Instead of: "Generate a test"

Mercury way: "You are a Mercury QA engineer. Generate xUnit tests for LoanService.ExtendLoan with NSubstitute mocks, following AAA pattern, including positive/negative/edge cases. If I have not given you enough information, please let me know what information is missing to make a better test."

Result:

✅ More detailed output

✅ Follows Mercury standards

✅ Includes all required patterns

Chain-of-Thought

What is Chain-of-Thought?

Ask Copilot to "think step by step" or "explain your reasoning"

Why It Works:

LLMs generate better output when they show their work

Trigger Phrases:

- "Let's think step by step"
- "Explain your reasoning"
- "Walk me through..."
- "Break this down into steps"
- "First, analyse... Then, suggest..."

Chain-of-Thought

Mercury Example: Architecture Decisions

❌ Basic: "Should I use repository pattern?"

✅ Chain-of-Thought:

"In the context of Mercury's loan processing system:

1. What are pros/cons of repository pattern?
2. What does our codebase currently use?
3. How would this affect our testing strategy?
4. What's your recommendation and why?"

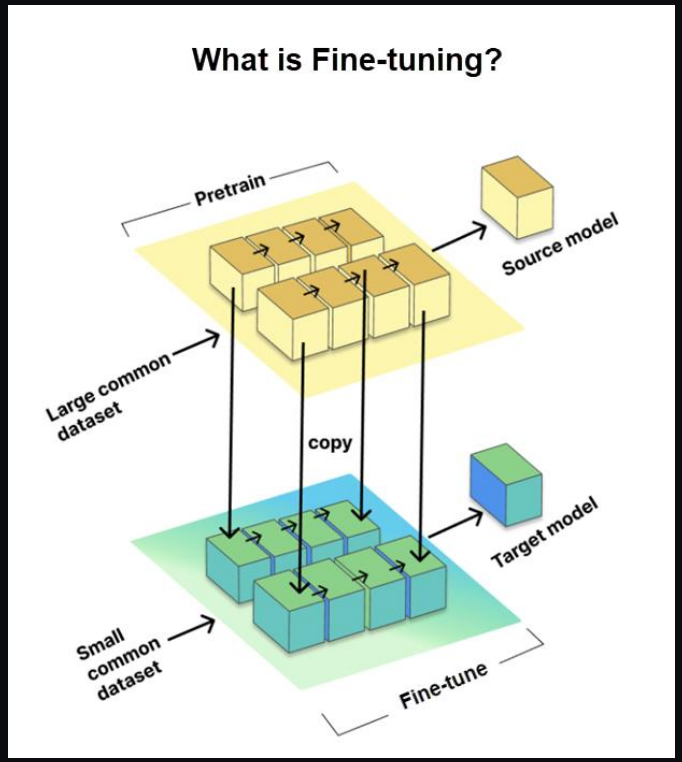
Result: Informed decision with reasoning

What are LLMs?

Large Language Models (LLMs) are artificial intelligence models designed and trained to understand, generate, and manipulate human language. These models are ingrained with the capability to handle a broad range of tasks involving text.

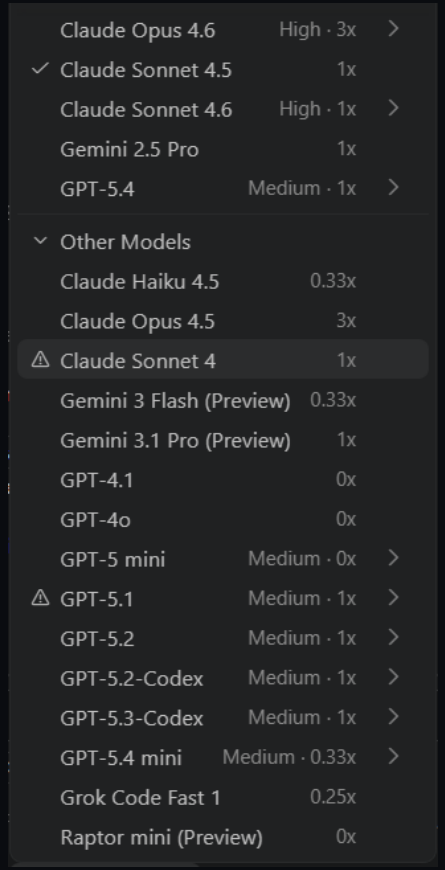
- Copilot

What are some core aspects?



- Volume of training data
- Contextual understanding
- Machine learning and AI integration
- Versatility through fine-tuning
- LoRA fine-tuning

Model Selection



- By default, Copilot Chat uses OpenAI's GPT-4.1 large language model.

- The following premium models are currently available in Copilot Chat:
  - Anthropic’s Claude Haiku 4.5
  - Anthropic’s Claude Sonnet 4.5 & 4.6
  - Anthropic’s Claude Opus 4.6
  - Google’s Gemini 2.5 Pro & 3.1 Pro
  - OpenAI’s GPT 5.4 & 5.4 mini & 5.3 codex
  - Grok Code Fast 1

Model Selection

- For balance between cost and performance, try GPT-5.3 Codex, 5.4 mini or Grok Code Fast 1
- For fast, low-cost support for simple repetitive tasks, try Claude Haiku 4.5.
- For deep reasoning or complex coding challenges, try GPT-5.4, Gemini 3.1 Pro or Claude Opus 4.5.
- For multimodal inputs (diagrams, screenshots or UI components) and questions about visual context, try Claude Sonnet 4.6, Gemini 3.1 Pro or GPT-5.4 mini.

Model Selection

- For balance between cost and performance, try GPT-5.3 Codex, 5.4 mini or Grok Code Fast 1
- For fast, low-cost support for simple repetitive tasks, try Claude Haiku 4.5.
- For deep reasoning or complex coding challenges, try GPT-5.4, Gemini 3.1 Pro or Claude Opus 4.5.
- For multimodal inputs (diagrams, screenshots or UI components) and questions about visual context, try Claude Sonnet 4.6, Gemini 3.1 Pro or GPT-5.4 mini.

Model Selection

Mercury Use Case Recommendations:

```
🏦 C# Backend Development
├─ Primary: GPT-5.3 Codex or GPT-5.4
│  └─ Why: Excellent with C#, async patterns, Entity Framework
├─ Alternative: Claude Sonnet 4.6
│  └─ Why: Strong reasoning for architecture decisions
└─ Budget: Claude Haiku 4.5 or GPT-5.4 mini
   └─ Why: Fast, good enough for simple CRUD
```

Model Selection

🧪 Test Generation
├─ Primary: GPT-5.4 or Claude Opus 4.6
│ └─ Why: Best at generating edge cases
└─ Budget: GPT-5.3 Codex
   └─ Why: Understands testing patterns well

📊 Data Engineering (Python/Spark)
├─ Primary: Gemini 3.1 Pro or GPT-5.4
│ └─ Why: Excellent with Python, data processing
└─ Budget: Claude Haiku 4.5
   └─ Why: Fast enough for DataFrame operations

Model Selection

🔄 DevOps (YAML/Scripts)
├─ Primary: Grok Code Fast 1 or GPT-5.3 Codex
│ └─ Why: Fast, good with config files
└─ Alternative: Claude Sonnet 4.6
   └─ Why: Good at explaining complex pipelines

📐 Architecture & Design Questions
├─ Primary: Claude Opus 4.6 or GPT-5.4
│ └─ Why: Deep reasoning, considers tradeoffs
└─ Alternative: Gemini 3.1 Pro
   └─ Why: Good multilingual understanding

Model Selection

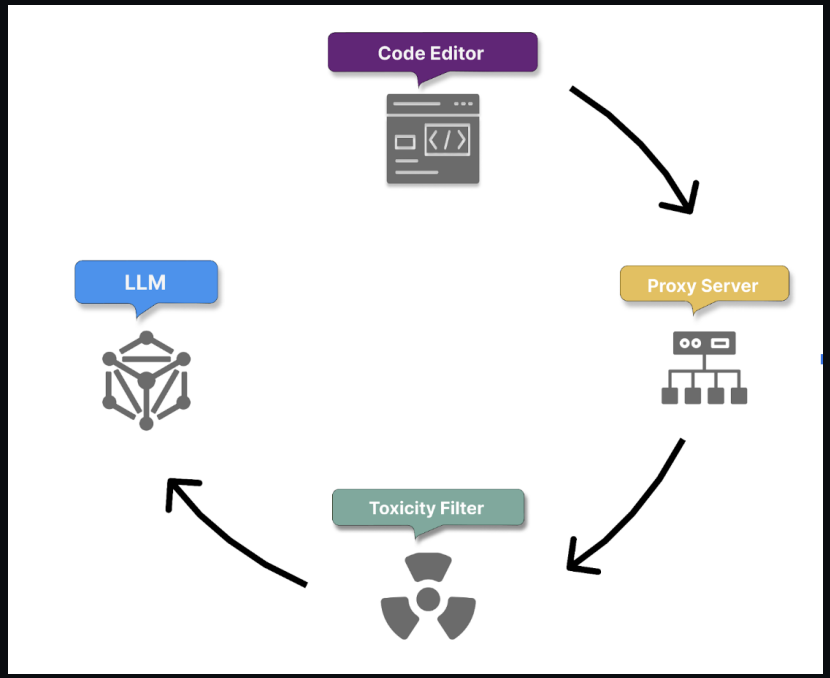
```
🖼️ Visual Context (Screenshots, Diagrams)
├─ Primary: Claude Sonnet 4.6 or Gemini 3.1 Pro
│   └─ Why: Best multimodal support
└─ Alternative: GPT-5.4 mini
    └─ Why: Budget-friendly multimodal
```

Model Selection

Pro Tips:

- 💡 Default (GPT-4.1) works well for most tasks
- 💡 Switch for complex reasoning (Opus) or speed (Haiku)
- 💡 Experiment! Different models for different tasks
- 💡 Cost vs. Performance: Start with defaults, upgrade if needed

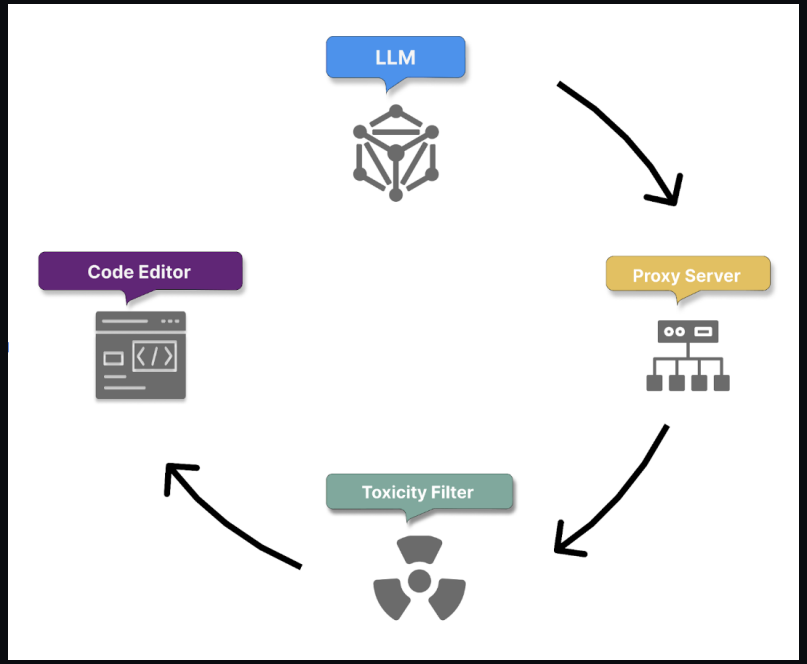
Copilot User Prompt Process Flow



Inbound flow:

1. Secure prompt transmission and context gathering
2. Proxy filter
3. Toxicity filtering
4. Code generation with LLM

Copilot User Prompt Process Flow



Outbound flow:

1. Post-processing and response validation
2. Suggestion delivery and feedback loop initiation
3. Repeat for subsequent prompts

Prompt types supported by GitHub Copilot Chat

- Direct Questions: *Why is my React component not rendering?*
- Code-Related Requests: *Fix this error in my code*
- Open-Ended Queries: *What are the best practices when implementing ORM?*
- Contextual Prompts: *Here's a part of my code, can you suggest improvements?*

Context Windows

- Context Window refers to the amount of surrounding code and text the model can process simultaneously to generate suggestions.
- Copilot Chat currently operates with a context window of 192k tokens*
- Be mindful of context window limitations when crafting prompts.
- Breaking down complex problems into smaller, more focused queries or providing relevant code snippets.

*Token is the basic unit of text that the model processes, acting as a building block for understanding and generating language.

Mercury Strategies to Save Context

- Start Fresh Conversations After 5-10 exchanges or when hitting limits
- Be Selective with #file and @codebase
- Use Custom Instructions Smartly
- Leverage Skills Over Instructions

Success Checklist

- Used 4S framework in at least 5 prompts
- Experimented with context variables (#file, @codebase)
- Tried at least 2 different models
- Apply at least 3 prompting techniques

Q & A

Upcoming Sessions

1. GitHub Copilot for Developers
2. GitHub Copilot for DevOps
3. Q&A Session

Thank you!